HOCHSCHULE ESSLINGEN

ProVaCaSy

Hochschule Esslingen
Software-Projekt SWB 4 – Wintersemester 2026/27

(Pro)VaCaSy

Professoren Vacation Calendar System

Meilenstein 1

von

Osman Sallamaci 781174
Tommy Luu 781294
Dominique Sluga 782866

Kunde:	Prof. Dr. Jörg Nitzsche
Betreuer:	Andreas Heinrich

Inhaltsverzeichnis

Abkürzungsverzeichnis **4**

1 Einführung **5**
1.1 Problemstellung . . . 5
1.2 Ansatz der Problemlösung . . . 5
1.3 Zielgruppe . . . 5
1.4 Personas . . . 5
1.4.1 Professor . . . 6
1.4.2 Sekretariat . . . 7
1.4.3 Dekanat . . . 8

2 Geforderter Funktionsumfang **9**
2.1 Funktionale Anforderungen MVP . . . 9
2.2 Nicht-funktionale Anforderungen . . . 9
2.3 Nice-To-Have . . . 10

3 UI-Entwürfe **10**
3.1 Mockup der Startseite . . . 10
3.2 Mockup der Statistikansicht des Dekanats . . . 10

4 Architektur **12**
4.1 Konzeptionelle Architektur . . . 12
4.2 Technologische Architektur . . . 12
4.2.1 Backend . . . 12
4.2.2 Frontend . . . 13
4.2.3 Datenbank . . . 13
4.2.4 Authentifizierung . . . 13
4.2.5 Betrieb (Docker, CI/CD) . . . 13
4.2.6 Künstliche Intelligenz (KI) . . . 13
4.3 Entwicklungsumgebung . . . 14
4.4 Lizenzen . . . 14

5 Projektmanagement **15**
5.1 Methode . . . 15
5.2 Rollen und Aufgabenverteilung . . . 16
5.3 Git-Workflow und Definition of Done . . . 16
5.4 Zeitplan und Meilensteine . . . 17
5.5 Risikomanagement . . . 19

6 Aufwandsschätzung **21**
6.1 Gruppenaufwand . . . 21
6.2 Individueller Aufwand . . . 22

Quellen **23**

Abbildungsverzeichnis

1 Prof. Dr. Klaus Günther Jost 6
2 Sekretärin - Anna Schmidt 7
3 Dekanat - Dr. Paul Hoffmann 8
4 Entwurf der Kalenderansicht 11
5 Entwurf der Statistikansicht des Dekanats 11
6 Kanban-Board in GitHub Projects vom 19.10.26 15

Tabellenverzeichnis

1 Lizenzen der eingesetzten Technologien 14
2 Rollen und Schwerpunkte im Entwicklungsteam 16
3 Termine und Inhalte der Meilensteine 18
4 Risiken und Gegenmaßnahmen 20
5 Aufwandsschätzung des Gruppenaufwands 21
6 Aufwandsschätzung je Aufgabe 22

Abkürzungsverzeichnis

(Pro)VaCaSy Professoren Vacation Calendar System

ORM Object-Relational Mapping

SPA Single Page Application

UI User Interface

CI/CD Continuous Integration / Continuous Deployment

MVP Minimum Viable Product

RBAC Role-Based Access Control

DTO Data Transfer Object

ICS iCalendar

IDE Integrated Development Environment

CSS Cascading Style Sheets

VS Code Visual Studio Code

SQL Structured Query Language

API Application Programming Interface

VW Vorlesungswochen

KI Küntliche Intelligenz

1 Einführung

Dieses Kapitel führt umfassend in das Projekt ein und beschreibt den Ausgangszustand sowie die daraus resultierende Problemstellung.

1.1 Problemstellung

Die Abwesenheiten der Professoren werden an der Fakultät bisher dezentral über Outlook als Kalendereintrag hinterlegt. Ein zentrales System, das Abwesenheiten und Urlaubstage gemeinsam verwaltet, existiert nicht. Daraus ergeben sich mehrere Probleme. Beim Eintragen kann es passieren, dass man doppelt bucht. Dies kann dazu führen, dass es als doppelter Urlaubstag gezählt wird. Zudem ist es nicht auf einen Blick erkennbar, wie viele der 30 Urlaubstage pro Jahr bereits verplant sind und wer sein Kontingent überschritten hat.

1.2 Ansatz der Problemlösung

Um die Schwachstellen des bisherigen Prozesses in Outlook zu beheben, entwickeln wir ein zentrales, webbasiertes Verwaltungssystem. Dieses bietet eine übersichtliche Oberfläche zur Erfassung und Nachverfolgung von Urlaubs- und Abwesenheitstagen. Das System berechnet das verbleibende Urlaubskontingent automatisch, verhindert Doppelbuchungen durch integrierte Validierungen und reduziert den manuellen Aufwand für die Verwaltung deutlich.

1.3 Zielgruppe

Die Zielgruppe der Anwendung setzt sich aus folgenden Rollen zusammen:

- **Professoren:** Nutzen das System zur Erfassung eigener Abwesenheiten, zur Einsicht in das verbleibende Urlaubskontingent und zur Orientierung über Abwesenheiten von Kollegen.
- **Sekretariat:** Behält den organisatorischen Überblick über alle anstehenden Abwesenheiten an der Fakultät.
- **Dekanat:** Übernimmt dieselben Aufgaben wie das Sekretariat und verfügt zusätzlich über die Berechtigung, Urlaubsanträge zu prüfen und zu genehmigen.

1.4 Personas

Die folgenden Personas repräsentieren die zentralen Nutzergruppen des Systems. Sie definieren deren spezifische Ziele, Bedürfnisse und Probleme, um die Anforderungen aus Anwendersicht greifbar zu machen.

1.4.1 Professor



Abbildung 1: Prof. Dr. Klaus Günther Jost[1]

Das ist Prof. Dr. Klaus Günther Jost, der Professor für Psychologie an einer Hochschule ist. Er ist 40 Jahre alt, zielorientiert und schätzt effiziente, digitale Abläufe im Hochschulalltag. Sein Terminkalender ist stark durch Vorlesungen, Forschungsaktivitäten und studentische Sprechstunden getaktet.

Ziele

- Schnelle und unkomplizierte Durchführung administrativer Aufgaben
- Lückenlose Übersicht über den eigenen Urlaubsanspruch und Abwesenheitsstatus
- Vermeidung von unnötigem Bürokratie- und Zeitaufwand im Lehrbetrieb

Bedürfnisse

- Eine benutzerfreundliche und übersichtliche Benutzeroberfläche
- Zuverlässige, zeitnahe Rückmeldungen und Benachrichtigungen
- Transparente Einsicht in Genehmigungsstände

Probleme

- Hoher Zeitverlust durch umständliche oder unübersichtliche Prozesse
- Unklarheit über verbleibende Urlaubstage und Bearbeitungsstände
- Fehlen einer zentralen und einfachen Lösung für Anträge

Nutzen durch die Applikation

Die (Pro)VaCaSy-Applikation könnte ihm helfen, Zeit bei der Beantragung und Verwaltung von Abwesenheiten zu sparen und administrative Abläufe zu vereinfachen. Durch direkte Statusanzeigen und übersichtliche Kontingentübersichten würde zudem die Transparenz erhöht. Darüber hinaus ermöglicht die Anwendung einen ortsunabhängigen und flexiblen Zugriff auf alle relevanten Daten, sodass er jederzeit den Überblick über seine Abwesenheiten behalten kann.

[1] Das Bild wurde KI-generiert von Gemini.

1.4.2 Sekretariat



Abbildung 2: Sekretärin - Anna Schmidt[2]

Das ist Anna Schmidt, eine Sekretärin an einer Hochschule. Sie ist 42 Jahre alt, zielorientiert und schätzt effiziente, digitale Abläufe im Hochschulalltag. Ihr Terminkalender ist stark durch die Koordination von Verwaltungsprozessen, die Bearbeitung von Anträgen und die organisatorische Unterstützung des Fachbereichs getaktet.

Ziele

- Effiziente Prüfung und Weiterleitung von Urlaubs- und Abwesenheitsanträgen
- Lückenlose Dokumentation aller Abwesenheiten im Fachbereich
- Minimierung des Koordinations- und Papieraufwands

Bedürfnisse

- Zentrales und übersichtliches Dashboard für alle Bereichsanträge
- Schnelle und transparente Freigabeprozesse
- Zuverlässige Benachrichtigungen bei Statusänderungen

Probleme

- Hoher Aufwand durch manuelle und unübersichtliche Antragswege
- Lange Wartezeiten und unklare Bearbeitungsstände
- Fehlende zentrale Übersicht über das Personal

Nutzen durch die Applikation

Früher war die Verwaltung von Abwesenheiten im Sekretariat oft von zeitaufwendigen Papieranträgen, unübersichtlichen E-Mail-Verläufen und manuellem Abgleich von Urlaubskontingenten geprägt. Die (Pro)VaCaSy-Applikation hilft ihr nun dabei, diese administrativen Abläufe im Fachbereich grundlegend zu vereinfachen und wertvolle Zeit zu sparen. Durch direkte Statusanzeigen, zentrale Verwaltung und übersichtliche Kontingentübersichten wird die Transparenz erheblich erhöht. Darüber hinaus ermöglicht die Anwendung einen ortsunabhängigen und flexiblen Zugriff auf alle relevanten Daten, sodass sie jederzeit den perfekten Überblick über die Abwesenheiten behalten kann.

[2]Das Bild wurde KI-generiert von Gemini.

1.4.3 Dekanat



Abbildung 3: Dekanat - Dr. Paul Hoffmann[3]

Das ist Dr. Paul Hoffmann, der dem Dekanat einer Hochschule angehört. Er ist 29 Jahre alt, zielorientiert und trägt die Verantwortung für die strategische Ausrichtung sowie die organisatorische und personelle Steuerung des Fachbereichs. Sein Arbeitsalltag ist geprägt von Gremiensitzungen, strategischen Entscheidungen und der finalen Freigabe von Anträgen.

Ziele

- Effiziente und rechtssichere Freigabe von übergeordneten Anträgen
- Gewährleistung einer lückenlosen Vertretungs- und Kapazitätsplanung
- Optimierung administrativer Entscheidungsprozesse auf Dekanatsebene

Bedürfnisse

- Zentrales Dashboard zur schnellen Prüfung und Freigabe
- Verlässliche Kennzahlen zur Personalverfügbarkeit im Fachbereich
- Transparente Historie und Nachvollziehbarkeit aller Freigaben

Probleme

- Verzögerungen durch intransparente oder papierbasierte Dienstwege
- Hoher Abstimmungs- und Nachfragebedarf bei Anträgen
- Mangelnde Gesamtübersicht über die personelle Auslastung

Nutzen durch die Applikation

Die (Pro)VaCaSy-Applikation hilft ihm dabei, diese übergeordnete Entscheidungsprozesse medienbruchfrei zu digitalisieren und zu beschleunigen. Durch digitale Freigabe-Workflows und direkte Statusanzeigen wird die organisatorische Effizienz erheblich gesteigert. Darüber hinaus ermöglicht die Anwendung einen ortsunabhängigen und flexiblen Zugriff auf alle relevanten Fachbereichsdaten, sodass er auch unterwegs oder zwischen Gremiensitzungen stets den vollen Überblick behält.

[3]Das Bild wurde KI-generiert von Gemini.

2 Geforderter Funktionsumfang

Dieses Kapitel legt fest, welche Funktionen die Anwendung bieten soll und welche bewusst nicht.

2.1 Funktionale Anforderungen MVP

Der für das Minimum Viable Product (MVP) geforderte Funktionsumfang umfasst sowohl systemweite als auch rollenspezifische Funktionen:

- **Allgemeine / Systemweite Funktionen:**
  - Zentraler und sicherer Login über Keycloak mit automatischer Rollenerkennung.
  - Dashboard zur Anzeige des persönlichen Urlaubskontingents und des Status eingereichter Anträge.
  - Bereitstellung von iCalendar (ICS) Dateien zur unkomplizierten Übernahme von Terminen in den Outlook-Kalender.
  - Erfassung, Bearbeitung und Löschung von Abwesenheitseinträgen.
  - Fachliche Differenzierung zwischen Urlaubs- und Konferenzeinträgen.
  - Automatische Prüfung auf zeitliche Überschneidungen (Doppelbuchungsprüfung)
- **Professor:**
  - Erstellen, Bearbeiten und Stornieren von eigenen Urlaubsanträgen und Abwesenheiten.
  - Einsicht in die Team-Kalenderübersicht zur Vermeidung von Terminüberschneidungen.
- **Sekretariat:**
  - Zentrale Einsicht in den Gesamtkalender aller Abwesenheiten an der Fakultät sowie der Statistiken.
- **Dekanat:**
  - Übernimmt dieselben Aufgaben wie das Sekretariat und besitzt zusätzlich die Berechtigung, Urlaubsanträge zu prüfen, zu genehmigen oder abzulehnen.

2.2 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen definieren die Qualitätsmerkmale, Rahmenbedingungen und technischen Standards des Systems:

- **Sicherheit:** Sichere Authentifizierung und Autorisierung über Keycloak sowie eine strikte rollenbasierte Zugriffskontrolle (Role-Based Access Control (RBAC)) zum Schutz sensibler Personaldaten. Darüber hinaus soll das System keine externen Ressourcen laden sondern alles lokal installiert.

- **Wartbarkeit & Qualität:** Einhaltung von Clean-Code-Prinzipien, automatisierte Code-Analysen über SonarQube sowie umfassende Unit- und Integrationstests in einer GitHub Actions Continuous Integration / Continuous Deployment (CI/CD) Pipeline.
- **Benutzerfreundlichkeit:** Ein intuitives, responsives User Interface (unter Verwendung von Vue.js und Tailwind CSS) für eine schnelle und unkomplizierte Bedienung. Die Anwendung wird standartmäßig in hellem design entwickelt.
- **Portabilität & Betrieb:** Vollständige Containerisierung der Anwendung mittels Docker und Docker Compose für einen standardisierten und reproduzierbaren Betrieb.

2.3 Nice-To-Have

Funktionen und Features, die über den geforderten Mindestumfang MVP hinausgehen und bei zeitlicher Verfügbarkeit optional umgesetzt werden können:

- **Outlook Extension:** Entwicklung einer plattformübergreifenden Outlook-Erweiterung zur komfortablen Verwaltung und Synchronisation direkt aus dem E-Mail-Programm heraus.
- **Statistiken und Datenexport:** Zusätzliche grafische Auswertungen der Abwesenheiten sowie Exportfunktionen für administrative Zwecke in pdf.

3 UI-Entwürfe

Dieses Kapitel zeigt die geplanten Oberflächen und erklärt, wie die Nutzergruppen mit der Anwendung arbeiten.

3.1 Mockup der Startseite

Die Startseite (Abbildung 4) enthält die Ansicht des gemeinsamen Urlaubs- und Abwesenheitskalenders mit den entsprechend eingetragenen Abwesenheiten. Links ist eine Menüleiste mit Buttons für die Startseite, Statistik, Urlaub beantragen, Profil und Logout. In der Kalenderansicht kann per Pfeilauswahl oder Auswahl des Datums den Monat wechseln und per Button zwischen der Monats-, Wochen- und Tagesansicht gewechselt werden.

3.2 Mockup der Statistikansicht des Dekanats

Die Statistikansicht des Dekanats (Abbildung 5) entspricht der des Sekretariats und zeigt eine Gesamtübersicht allen Urlaubs, aufgeteilt in Gesamtanspruch, genommenem Urlaub und Resturlaub, sowie einer Detailsansicht, welcher Professor in welchem Monat wieviel Urlaub genommen hat. Zudem ist eine Gesamtübersicht des Urlaubs und der Gesamtabwesenheit jedes Professors zu sehen. In dieser Ansicht kann die Jahresstatistik durchgewechselt werden. Zudem kann im Falle des Dekans dieser zwischen seiner eigenen Statistik und der Gesamtstatistik wechseln, sowie die Daten als CSV-Datei exportieren.

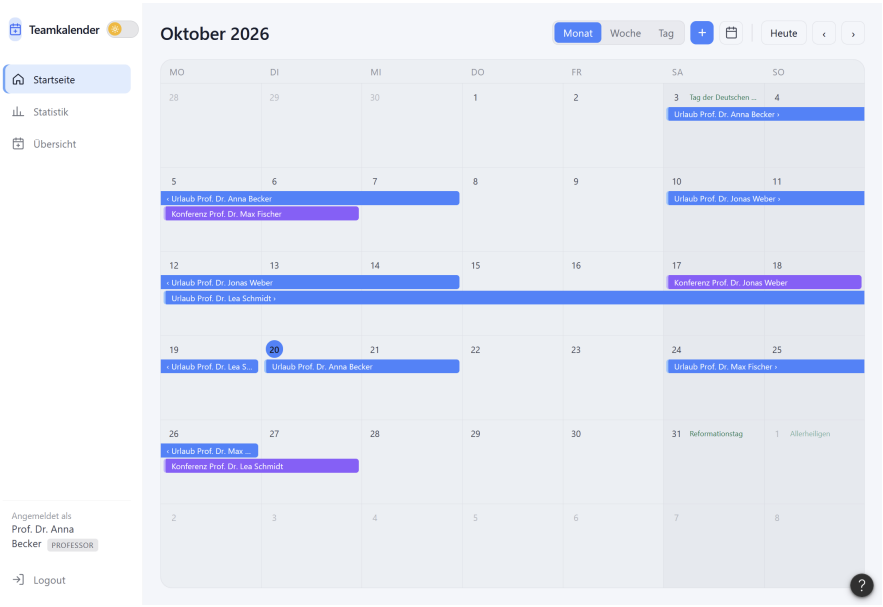


Abbildung 4: Entwurf der Kalenderansicht

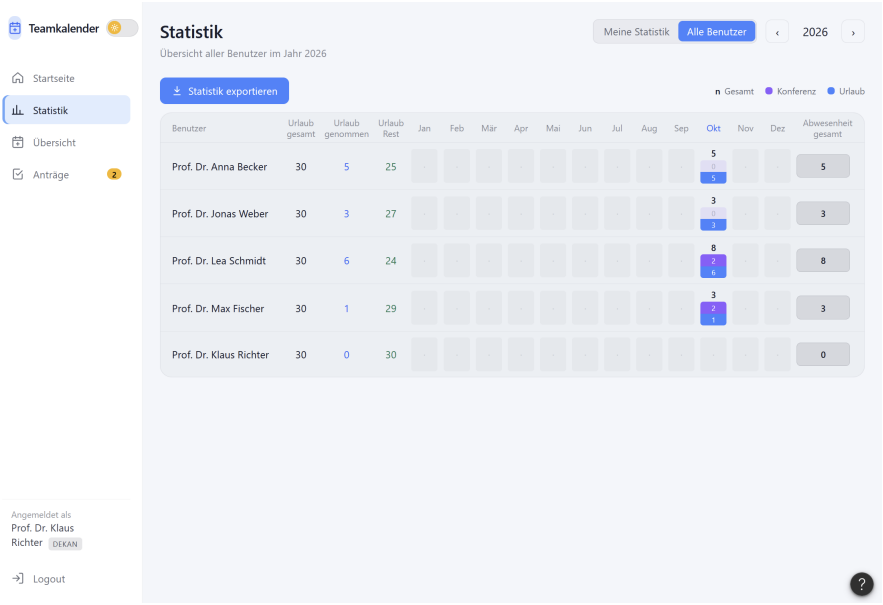


Abbildung 5: Entwurf der Statistikansicht des Dekanats

Hier ist der Stand des kompletten Figma Mockups einzusehen:
`https://www.figma.com/make/1AZNUv3UXnG95ia6faE5eR/Kalender-Webapplikation-erstellen?t=uXFBsN2wk0bCgZ6T-20&fullscreen=1`

4 Architektur

Dieses Kapitel beschreibt den Aufbau des Systems konzeptionell und die gewählten Technologien samt Begründung.

4.1 Konzeptionelle Architektur

Die konzeptionelle Architektur von *(Pro)VaCaSy* folgt einer klassischen 3-Schichten-Architektur (3-Tier-Architecture), um eine klare Trennung der Verantwortlichkeiten zwischen Benutzeroberfläche, Geschäftslogik und Datenhaltung zu gewährleisten:

- **Presentation Layer (Frontend):** Die Benutzeroberfläche als Single-Page Application SPA, die für die Darstellung der Kalenderansichten, Dashboards und Eingabemasken zuständig ist und mit dem Backend kommuniziert.
- **Business Logic Layer (Backend):** Das Backend enthält die gesamte Fachlogik. Es prüft Urlaubsanträge und berechnet das Urlaubskontingent. Außerdem steuert es Rollen und Berechtigungen. Intern ist es in Controller, Services und Repositories getrennt. Für den Datenaustausch kommen Data Transfer Object (DTO) zum Einsatz.
- **Data Access Layer (Datenbank):** Die relationale Datenbank zur dauerhaften und konsistenten Speicherung von Benutzerdaten, Urlaubsanträgen und Systemkonfigurationen.

4.2 Technologische Architektur

Die technologische Architektur beschreibt den gewählten Software-Stack sowie die eingesetzten Werkzeuge für die Umsetzung, den Betrieb und die Qualitätssicherung der Anwendung.

4.2.1 Backend

Das Backend von *(Pro)VaCaSy* wird auf Basis von Java Spring Boot implementiert und folgt einer strikten Schichtentrennung, bei der Controller nicht direkt auf die Datenbank zugreifen. Zur klaren Kapselung werden DTOs eingesetzt, um die übertragenen Daten gezielt von den internen Datenbankmodellen zu trennen. Die Qualität des Systems wird durch automatisierte Unit- und Integrationstests kontinuierlich abgesichert.

4.2.2 Frontend

Das Frontend von *(Pro)VaCaSy* wird als moderne Single Page Application (SPA) auf Basis von Vue.js realisiert. Es ist für die intuitive Darstellung der Benutzeroberfläche zuständig, einschließlich der Kalenderansichten, Dashboards und Eingabemasken für Urlaubsanträge. Für das Design und Layout wird Tailwind CSS eingesetzt. Die Kommunikation mit dem Backend erfolgt über definierte Schnittstellen, um Daten asynchron und performant auszutauschen.

4.2.3 Datenbank

Als Datenbanksystem kommt Postgre SQL zum Einsatz. Es speichert alle Daten dauerhaft und strukturiert. Dazu gehören Benutzerdaten, Urlaubsanträge und Abwesenheitszeiten. Das Spring-Boot-Backend greift über Object-Relational Mapping (ORM) auf die Datenbank zu. So bleiben die Datenzugriffe sauber gekapselt.

4.2.4 Authentifizierung

Für die Authentifizierung und Autorisierung wird Keycloak als zentraler Identitäts- und Access-Management-Dienst eingesetzt. Keycloak übernimmt die sichere Verwaltung von Benutzeridentitäten sowie die Verteilung von Tokens. Die Autorisierung basiert auf einer strikten RBAC, wodurch sichergestellt wird, dass Benutzer je nach ihrer Rolle (Professor, Sekretariat, Dekanat) ausschließlich Zugriff auf die für sie autorisierten Funktionen und Daten erhalten.

4.2.5 Betrieb (Docker, CI/CD)

Für den standardisierten und reproduzierbaren Betrieb wird die Anwendung vollständig mittels Docker und Docker Compose containerisiert. Die Services werden in isolierten Umgebungen ausgeführt, um Abweichungen zwischen Entwicklungs- und Produktionsumgebungen zu minimieren. Zur Automatisierung des Entwicklungs- und Testprozesses wird eine GitHub Actions CI/CD-Pipeline eingesetzt. Diese führt bei Änderungen automatisch Code-Analysen über SonarQube, Unit- und Integrationstests sowie das Bauen der Anwendung durch. Die erstellten Docker-Images werden über GitHub Packages innerhalb der Projektorganisation verwaltet.

4.2.6 Künstliche Intelligenz (KI)

Zur Effizienzsteigerung im Entwicklungsprozess und bei der Dokumentationserstellung kommen zwei spezialisierte KI-Systeme zum Einsatz:

- **Claude (Anthropic):** Dient als Unterstützung bei der Implementierung, dem Refactoring von Quellcode sowie bei der Lösung technischer Problemstellungen im Backend- und Frontend-Bereich.

- **Gemini (Google):** Wird gezielt zur sprachlichen Überarbeitung, Optimierung des Satzbaus und Qualitätssicherung der deutschen Texte innerhalb der Dokumentation genutzt.

4.3 Entwicklungsumgebung

Für die Entwicklung von *(Pro)VaCaSy* wird Visual Studio Code (VS Code) als einheitliche und erweiterbare Integrated Development Environment (IDE) eingesetzt. Dank einer modularen Architektur und spezifischer Erweiterungen bietet VS Code eine optimale Unterstützung für den gesamten verwendeten Technologie Stack:

- **Backend-Entwicklung:** Unterstützung für Java und Spring Boot durch dedizierte Erweiterungen für Code-Vervollständigung, Refactoring und strukturiertes Debugging.
- **Frontend-Entwicklung:** Bereitstellung von Werkzeugen für Vue.js, JavaScript/TypeScript sowie für das Styling mit Tailwind CSS.
- **Qualitätssicherung:** Direkte Einbindung von Werkzeugen zur lokalen Code-Analyse zur frühzeitigen Identifikation von Fehlern und Code Smells während der Implementierung.
- **Versionierung & Containerisierung:** Nahtlose Integration von Git für die Versionsverwaltung sowie Docker-Erweiterungen zur lokalen Steuerung und Überprüfung der Container-Umgebungen.

4.4 Lizenzen

Tabelle 1 gibt einen Überblick über die eingesetzten Open-Source-Technologien und deren jeweilige Lizenzmodelle. Da sämtliche Komponenten kostenfrei nutzbar sind, entstehen dem Projekt keine Lizenzkosten.

Technologie	**Einsatz**	**Lizenz**
Spring Boot	Backend	Apache 2.0 Lizenz
Vue.js	Frontend	MIT-Lizenz
Tailwind CSS	Styling	MIT-Lizenz
PostgreSQL	Datenbank	PostgreSQL-Lizenz
Keycloak	Authentifizierung	Apache 2.0 Lizenz

Tabelle 1: Lizenzen der eingesetzten Technologien

5 Projektmanagement

Dieses Kapitel beschreibt, wie das Team organisiert ist und arbeitet.

5.1 Methode

Das Team arbeitet agil nach dem Kanban-Prinzip, ergänzt um einzelne Elemente aus Scrum, nämlich regelmäßige Abstimmungstermine und eine fortlaufend priorisierte Aufgabenliste (Backlog). Alle Anforderungen, Aufgaben und Fehler werden als Issues im GitHub-Projekt der Organisation *(Pro)VaCaSy* erfasst und auf einem gemeinsamen Kanban-Board dargestellt (siehe Abbildung 6). Das Board besteht aus den Spalten *Backlog*, *Todo*, *In Progress*, *Review*, *Testing* und *Done*. Eine Aufgabe wird erst nach *Done* verschoben, wenn sie die Definition of Done (Abschnitt 5.3) erfüllt. Damit niemand überlastet wird und Aufgaben nicht liegen bleiben, bearbeitet jedes Teammitglied nur wenige Aufgaben gleichzeitig.

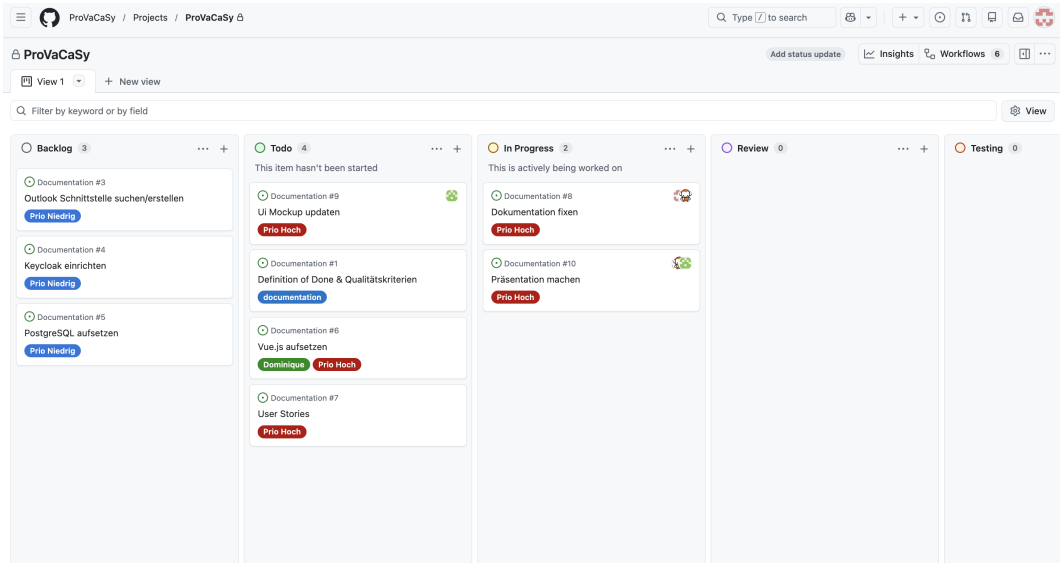


Abbildung 6: Kanban-Board in GitHub Projects vom 19.10.26

Der Aufwand wird direkt an den Issues erfasst. Jedes Issue enthält eine Schätzung in Stunden und die tatsächlich aufgewendete Zeit. So lassen sich die Schätzungen aus Kapitel 6 laufend mit dem realen Aufwand vergleichen.

Die Abstimmung mit den Beteiligten erfolgt nach folgendem Rhythmus:

- **Betreuermeeting:** einmal pro Woche mit dem Betreuer Andreas Heinrich, wahlweise an der Hochschule oder online über WebEx. Dort wird der Stand vorgestellt und das weitere Vorgehen abgestimmt.
- **Teammeetings:** zweimal pro Woche intern über Discord, um Aufgaben zu verteilen, Blockaden zu lösen und das Kanban-Board zu aktualisieren.

- **Kundenmeetings:** wöchentlich mit dem Kunden Prof. Dr. Jörg Nitzsche online über WebEx oder an der Hochschule, um Anforderungen zu klären und gemeinsam zu priorisieren.

5.2 Rollen und Aufgabenverteilung

Die Projektbeteiligten haben folgende Rollen: Prof. Dr. Jörg Nitzsche ist der Kunde und legt die fachlichen Anforderungen fest. Andreas Heinrich betreut das Projekt und begleitet es aus Sicht der Hochschule. Das Entwicklungsteam besteht aus drei Mitgliedern. Tabelle 2 zeigt deren Schwerpunkte.

Die Schwerpunkte benennen jeweils die Person, die für einen Bereich verantwortlich ist und ihn vorantreibt. Sie schließen die Mitarbeit der anderen nicht aus. Dank des Vier-Augen-Prinzips (Abschnitt 5.3) sieht jede Änderung mindestens eine zweite Person, wodurch das Wissen im Team verteilt bleibt.

Teammitglied	**Verantwortungsbereiche**
Dominique Sluga	Frontend
Tommy Luu	Backend, Keycloak
Osman Sallamaci	Backend, Docker

Tabelle 2: Rollen und Schwerpunkte im Entwicklungsteam

5.3 Git-Workflow und Definition of Done

Der Quellcode liegt in der GitHub-Organisation *(Pro)VaCaSy* in getrennten Repositories für Backend, Frontend und Dokumentation. Das Team arbeitet nach dem Feature-Branch-Workflow:

1. Für jede Aufgabe existiert ein Issue. Daraus entsteht ein eigener Branch, zum Beispiel `feature/12-kalender`.
2. Änderungen werden in kleinen, in sich abgeschlossenen Commits mit aussagekräftiger Nachricht gespeichert (Konvention: *Conventional Commits*).
3. Danach wird ein Pull Request auf den Branch `main` geöffnet und mit dem Issue verknüpft.
4. Die automatische Pipeline baut das Projekt, führt die Tests aus und analysiert den Code mit SonarQube.
5. Mindestens ein anderes Teammitglied prüft den Pull Request (Vier-Augen-Prinzip).
6. Erst nach bestandener Pipeline und erteilter Freigabe wird in `main` gemergt. Der Branch `main` ist geschützt, direkte Änderungen sind nicht möglich.

Eine Aufgabe gilt als abgeschlossen, wenn alle folgenden Punkte der Definition of Done erfüllt sind:

Entwicklung Die Anforderungen und Akzeptanzkriterien des Issues sind umgesetzt. Der Code folgt den Clean-Code-Prinzipien und der Schichtentrennung aus Abschnitt 4.2.1.

Tests Für neue Funktionen existieren Unit- und Integrationstests, und die Pipeline läuft fehlerfrei durch. Das Frontend wird zusätzlich anhand eines manuellen Testskriptbuchs geprüft.

Dokumentation Relevante Änderungen sind dokumentiert. Dazu gehören die API-Beschreibung und, wo nötig, die LaTeX-Dokumentation.

Vier-Augen-Prinzip Ein weiteres Teammitglied hat die Änderung geprüft und freigegeben.

Aufwand Die aufgewendete Zeit ist im Issue eingetragen.

5.4 Zeitplan und Meilensteine

Das Projekt läuft im Wintersemester 2026/27 und folgt dem Terminplan der Lehrveranstaltung. Dieser gibt fünf Abgaben beziehungsweise Termine vor und endet mit der finalen Abgabe am 19.01.2027. Tabelle 3 zeigt die Termine und die jeweils geforderten Inhalte. Die Vorlesungswochen (VW) beziehen sich auf den Semesterplan.

Das Team ordnet seine Arbeitspakete aus Kapitel 6 diesen Terminen in drei Phasen zu. Die Phasen sind eine Orientierung und keine starre Vorgabe, da sich Prioritäten im Projektverlauf ändern können.

Phase 1 (bis Meilenstein 2) Anforderungen klären, UI-Mockups und User Stories erstellen, das technische Konzept ausarbeiten und die Projektumgebung einrichten (GitHub-Organisation, Repositories, Definition of Done).

Phase 2 (bis Meilenstein 3) Den Prototyp mit Keycloak-Login, Datenbank, Grundgerüst des Frontends und CI/CD-Pipeline aufbauen. Damit steht früh eine durchgängige Basis, auf der die weiteren Funktionen entstehen.

Phase 3 (bis zur finalen Abgabe) Die Funktionen des MVP umsetzen, nämlich Urlaubsanträge, Kalender, Genehmigung durch das Dekanat sowie E-Mail mit ICS-Datei. Außerdem werden Tests ergänzt, das Installations- und Administrationshandbuch geschrieben und die Dokumentation abgeschlossen.

Die Nice-to-Have-Funktionen aus Abschnitt 2.3 werden erst eingeplant, wenn das MVP vollständig umgesetzt ist. Weicht der tatsächliche Projektstand vom Plan ab, dokumentiert das Team die Abweichung, begründet sie kurz und passt die weitere Planung an. Diese Gegenüberstellung von Planung und tatsächlichem Verlauf fließt in die Reflexion der finalen Dokumentation ein.

Meilenstein	**Termin**	**Inhalt**
Meilenstein 1	20.10.2026 23:59 Uhr (VW 4)	Konzeptdokument mit Problemstellung, Zielgruppe, Funktionsumfang, UI-Entwürfen, Architektur, Projektmanagement und Aufwandsschätzung (dieses Dokument)
Zwischenpräsentation	27.10.2026 09:45 Uhr (VW 5)	Präsentation (20 Minuten inklusive Diskussion) zu Anforderungen, Architektur, Technologien, Entwicklungsumgebung, Nutzerinteraktion und Zusammenarbeit im Team
Meilenstein 2	17.11.2026 23:59 Uhr (VW 8)	Erweitertes Dokument mit Product Vision, User Stories, UI-Mockups und technischem Konzept (Struktur-, Verhaltens- und Verteilungssicht, Schnittstellenbeschreibung, logisches und physisches Datenmodell)
Meilenstein 3	08.12.2026 23:59 Uhr (VW 11)	Überarbeitetes Dokument und vorführbarer Prototyp inklusive Login
Code Reviews	VW 12	Jedes Teammitglied präsentiert einen Teil des Codes
Finale Abgabe	19.01.2027 23:59 Uhr (VW 15)	Abschlusspräsentation und vollständige Dokumentation mit Installations- und Administrationshandbuch, Aufteilung des Teams, Reflexion, Lizenzen und Ausblick

Tabelle 3: Termine und Inhalte der Meilensteine

5.5 Risikomanagement

Dieser Abschnitt benennt die wichtigsten Projektrisiken und beschreibt, wie das Team ihnen begegnet.

Die Risiken wurden nach Eintrittswahrscheinlichkeit und Auswirkung bewertet (Tabelle 4). Sie werden in den Teammeetings regelmäßig überprüft und bei Bedarf angepasst.

Risiko	**Wahrsch.**	**Auswirkung**	**Gegenmaßnahme**
Ausfall eines Teammitglieds (z. B. Krankheit)	mittel	hoch	Wissen verteilen durch Vier-Augen-Prinzip, Entscheidungen dokumentieren, Aufgaben nicht an eine Person binden
Aufwand unterschätzt (z. B. Kalender, E-Mail und ICS)	hoch	mittel	MVP strikt priorisieren, Nice-to-Have zuletzt, Schätzung regelmäßig mit erfasster Zeit abgleichen
Anforderungen unklar oder ändern sich	mittel	mittel	Wöchentliche Abstimmung mit dem Betreuer, Kundenmeetings, Anforderungen als User Stories mit Akzeptanzkriterien
Keycloak-Anbindung gelingt nicht wie geplant	mittel	hoch	Früh im Prototyp integrieren, lokale Keycloak-Instanz in Docker Compose für die Entwicklung
Wissenslücken bei Spring Boot, Vue.js oder Docker	mittel	mittel	Einarbeitung früh einplanen, gegenseitige Unterstützung, bestehende Projekte der Hochschule als Referenz nutzen
Integrationsprobleme zwischen Frontend und Backend	mittel	mittel	OpenAPI-Spezifikation als Schnittstellenvertrag, Schnittstellen vor der Umsetzung abstimmen, Integrationstests in der Pipeline
Verletzung des Datenschutzes bei Personaldaten	gering	hoch	Rollenbasierte Zugriffskontrolle über Keycloak, nur notwendige Daten speichern, keine Echtdaten in Test- und Entwicklungsumgebungen
Sinkende Codequalität	mittel	mittel	SonarQube-Quality-Gate in der Pipeline, Code-Reviews, automatisierte Tests
Zeitliche Überlastung durch andere Module	hoch	mittel	Pufferzeiten einplanen, Meilensteine rechtzeitig vor der Prüfungsphase legen

Tabelle 4: Risiken und Gegenmaßnahmen

6 Aufwandsschätzung

Dieses Kapitel schätzt den Aufwand je Funktion; die Gesamtsumme wird mit dem Zeitbudget verglichen.

6.1 Gruppenaufwand

Tabelle 5 gibt einen Überblick über den geforderten zeitlichen Gesamtaufwand sowie die verbindlichen Aufwandsschätzungen für die Gruppenmitglieder im Rahmen des Projekts.

Aufgabe / Posten	**Summe (h)**
Kundenmeetings	
15 VW × 0,5 h	7,5
Betreuermeetings	
15 VW × 1,0 h	15,0
Präsentation und Disputation	
inkl. Vor- und Nachbereitung	10,0
Teambildung und Konfliktlösung	
inkl. Vor- und Nachbereitung	10,0
Zwischenpräsentation	
inkl. Vor- und Nachbereitung	10,0
Gruppenmeetings	
15 VW × 2,0 h	30,0
Gesamtaufwand pro Person	**82,5**

Tabelle 5: Aufwandsschätzung des Gruppenaufwands

6.2 Individueller Aufwand

Neben den allgemeinen organisatorischen Gruppenaufgaben fällt der Hauptteil des zeitlichen Aufwands auf die individuelle Konzeption, Implementierung und das Testing der fachlichen Anforderungen. Tabelle 6 gibt eine detaillierte Übersicht über die geschätzten Aufwände der einzelnen Features und Funktionsbausteine des Systems.

Aufgabe	Code (h)	Test (h)	Doku (h)	Summe (h)
Backend				
Keycloak	15	5	5	25
Datenbank	40	10	5	55
Kalender	40	10	5	55
E-Mail und ICS	50	10	5	65
Statistiken	15	5	5	25
Summe Backend	160	40	25	225
Frontend				
Login	15	5	5	25
Grundgerüst	70	20	10	100
Kalender	30	10	5	45
Statistiken	15	5	5	25
Summe Frontend	130	40	25	195
Sonstiges				
Docker	15	5	5	25
Github CI/CD	20	5	5	30
Sonarqube	10	5	5	20
Reviews	20	0	0	20
Summe Sonstiges	65	15	15	95
Zwischensumme	**355**	**95**	**65**	**515**

Tabelle 6: Aufwandsschätzung je Aufgabe

Quellen

Hochschule Esslingen Logo